

SQL NEW QUESTION 2 — Event Average Finish Time

Database: ATHLETICS_RESULTS_DB

TABLE 1 — events

Table Structure

Column	Data Type	Key	Description
event_id	INT	PK	Unique event ID
event_name	VARCHAR(100)	—	Name of the event

CREATE TABLE Query

```
CREATE TABLE events (  
  event_id INT PRIMARY KEY,  
  event_name VARCHAR(100)  
);
```

INSERT Query

```
INSERT INTO events VALUES  
(1, '100m Sprint'),  
(2, '200m Sprint');
```

Inserted Records

event_id	event_name
1	100m Sprint
2	200m Sprint

TABLE 2 — athletes

Table Structure

Column	Data Type	Key	Description
athlete_id	INT	PK	Unique athlete ID
athlete_name	VARCHAR(100)	—	Name of the athlete

CREATE TABLE Query

```
CREATE TABLE athletes (  
  athlete_id INT PRIMARY KEY,  
  athlete_name VARCHAR(100)  
);
```

INSERT Query

```
INSERT INTO athletes VALUES  
(1, 'Alice'),  
(2, 'Brian'),  
(3, 'Carlos'),  
(4, 'Diana');
```

Inserted Records

athlete_id	athlete_name
1	Alice
2	Brian
3	Carlos
4	Diana

TABLE 3 — results

Table Structure

Column	Data Type	Key	Description
result_id	INT	PK	Unique result ID
event_id	INT	FK	Event in which athlete participated
athlete_id	INT	FK	Athlete who participated
finish_time	DECIMAL(5,2)	—	Finish time in seconds
status	ENUM('Recorded','Disqualified','DNS')	—	Status of the result

CREATE TABLE Query

```
CREATE TABLE results (  
  result_id INT PRIMARY KEY,  
  event_id INT,  
  athlete_id INT,  
  finish_time DECIMAL(5,2),  
  status ENUM('Recorded','Disqualified','DNS')  
);
```

INSERT Query

```
INSERT INTO results VALUES  
(201, 1, 1, 11.20, 'Recorded'),  
(202, 1, 2, 10.95, 'Recorded'),  
(203, 1, 3, 12.10, 'Recorded'),  
(204, 1, 4, 11.50, 'Disqualified'),  
(205, 2, 1, 22.50, 'Recorded'),  
(206, 2, 2, 21.80, 'Recorded'),  
(207, 2, 3, 21.80, 'Recorded'),  
(208, 2, 4, 23.10, 'DNS');
```

RESULT QUERY — Event Average Finish Time

```
SELECT e.event_name, a.athlete_name, r.finish_time,  
ROUND(AVG(r.finish_time) OVER (PARTITION BY r.event_id), 2) AS avg_event_time  
FROM results r  
JOIN events e ON r.event_id = e.event_id  
JOIN athletes a ON r.athlete_id = a.athlete_id  
WHERE r.status = 'Recorded'  
ORDER BY e.event_name, r.finish_time, a.athlete_name;
```

Expected Output

event_name	athlete_name	finish_time	avg_event_time
100m Sprint	Brian	10.95	11.42
100m Sprint	Alice	11.20	11.42
100m Sprint	Carlos	12.10	11.42
200m Sprint	Brian	21.80	22.03
200m Sprint	Carlos	21.80	22.03
200m Sprint	Alice	22.50	22.03

SQL NEW QUESTION 1 — Days Between Customer Orders

Database: CUSTOMERS_ORDERS_DB

TABLE 1 — customers

Table Structure

Column	Data Type	Key	Description
customer_id	INT	PK	Unique customer ID
customer_name	VARCHAR(100)	—	Customer name

CREATE TABLE Query

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY,  
  customer_name VARCHAR(100)  
);
```

INSERT Query

```
INSERT INTO customers VALUES  
(1, 'Aarav'),  
(2, 'Diya'),  
(3, 'Kabir');
```

Inserted Records

customer_id	customer_name
1	Aarav
2	Diya
3	Kabir

TABLE 2 — orders

Table Structure

Column	Data Type	Key	Description
order_id	INT	PK	Unique order ID
customer_id	INT	FK	Customer who placed the order
order_date	DATE	—	Date when order was placed
amount	DECIMAL(10,2)	—	Order amount

CREATE TABLE Query

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT,  
  order_date DATE,  
  amount DECIMAL(10,2)  
);
```

INSERT Query

```
INSERT INTO orders VALUES  
(501, 1, '2025-08-02', 800.00),  
(506, 1, '2025-08-07', 1200.00),  
(514, 1, '2025-08-19', 950.00),  
(602, 2, '2025-08-04', 650.00),  
(609, 2, '2025-08-13', 1050.00),  
(701, 3, '2025-08-06', 1500.00),  
(708, 3, '2025-08-16', 1100.00);
```

RESULT QUERY — Days Between Customer Orders

```
SELECT c.customer_name, o.order_id, o.order_date,
DATEDIFF(o.order_date, LAG(o.order_date) OVER (
    PARTITION BY o.customer_id
    ORDER BY o.order_date, o.order_id
)) AS days_since_previous_order
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
ORDER BY c.customer_name, o.order_date, o.order_id;
```

Expected Output shown in PDF

customer_name	order_id	order_date	days_since_previous_order
Aditi	101	2025-11-07T00:00:00.000Z	NULL
Aditi	102	2025-11-11T00:00:00.000Z	4
Aditi	105	2025-11-15T00:00:00.000Z	4
Bharat	201	2025-11-14T00:00:00.000Z	NULL

Source note: The PDF's sample data and expected-output image contain different records; both are retained as shown in the source.